

EVENTSPHERE - ONLINE EVENT REGISTRATION SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

M.N SAHITHI KIRAN (VTU24654)

*10211CS224
Full Stack Application Development
Slot : E2-B20*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "EventSphere - Online Event Registration System" by M.N SAHITHI KIRAN (VTU24654) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. S.HEMAMALINI ME.,PH.D.

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-ordinator

Dr.Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(M.N SAHITHI KIRAN)

Date:06/05/2026

ABSTRACT

TheEventSphere - Online Event Registration System is a web-based application developed to streamline and automate the management of events within a college campus. In many educational institutions, organizing campus events such as seminars, workshops, cultural programs, technical fests, and club activities is often handled manually, which can lead to communication gaps, scheduling conflicts, and inefficient coordination. This project aims to provide a centralized digital platform to simplify the entire event management process.

The system allows event organizers to create, update, and manage events by entering details such as event name, date, time, venue, category, and description. Students can view upcoming events, register online, and receive instant notifications regarding event updates or changes. Administrators can monitor all events, manage users, approve event requests, and maintain event records efficiently.

The EventSphere - Online Event Registration System improves communication between students, faculty, and organizers by offering a structured and transparent platform for event coordination. It reduces paperwork, saves time, minimizes manual errors, and ensures better

scheduling and resource management. The system also enhances student participation by providing easy access to event information and registration facilities.

This project is developed using modern web technologies with a user-friendly interface to ensure accessibility and ease of use. The proposed system provides an efficient, reliable, and smart solution for managing campus events digitally, making event organization more effective and organized in educational institutions.

Keywords: EventSphere, Online Event Registration, Web Application, Student Registration, Event Scheduling, Notifications, Campus Automation, Centralized Platform

LIST OF FIGURES

1.1	System Overview	4
3.1	Frontend and Backend Workflow	9
5.1	System Output Screens	17

LIST OF TABLES

5.1	Comparison of Existing System and Proposed System .	15
-----	---	----

LIST OF ACRONYMS AND ABBREVIATIONS

ESOERS	EventSphere - Online Event Registration System
UI	User Interface
UX	User Experience
DBMS	Database Management System
SQL	Structured Query Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
PHP	Hypertext Preprocessor
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ERD	Entity Relationship Diagram

HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
OTP	One-Time Password
SMS	Short Message Service
PDF	Portable Document Format
ID	Identification
Admin	Administrator

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ACRONYMS AND ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	5
3 FRAMEWORK DESCRIPTION	7
3.1 Frontend Implementation	7

3.2	Backend Implementation	8
3.3	Database Implementation	8
4	METHODOLOGIES	11
4.1	Project Architecture	11
4.2	DataFlow Diagram	11
4.3	Module 1: User Management	12
4.4	Module 2: Event Management	12
4.5	Module 3: Notification Management	13
5	RESULTS AND DISCUSSIONS	15
6	CONCLUSION AND FUTURE ENHANCEMENTS	18
6.1	Conclusion	18
6.2	Future Enhancements	18
	References	19

Chapter 1

INTRODUCTION

1.1 Introduction

The EventSphere - Online Event Registration System is a web-based application developed to simplify and automate the process of organizing and managing events within an educational institution. In colleges and universities, various events such as seminars, workshops, technical symposiums, cultural festivals, placement drives, and club activities are conducted regularly. Managing these events manually often leads to communication gaps, scheduling conflicts, delayed registrations, and inefficient coordination among students, faculty, and organizers.

The proposed system provides a centralized digital platform where event organizers can create and manage events, students can register and receive notifications, and administrators can monitor and control the entire process. This system ensures better communication, im-

proved scheduling, and efficient event coordination across the campus.

1.2 Aim of the Project

The main aim of the EventSphere - Online Event Registration System is to develop a centralized and user-friendly platform for efficiently managing campus events. The project aims to reduce manual effort, improve communication between stakeholders, automate registration and scheduling, and enhance student participation through a smart digital solution.

1.3 Scope of the Project

The scope of this project includes digital event creation, scheduling, participant registration, user management, event approval, notifications, and report generation. The system can be used by students, faculty members, event coordinators, and administrators within a college campus. It can be extended in the future to include mobile applications, payment integration, attendance tracking, and AI-based event recommendations.

1.4 Framework

TheEventSphere - Online Event Registration System is developed using a modern web development framework. The frontend is built using HTML, CSS, and JavaScript to provide an interactive user interface. The backend is developed using PHP to handle server-side operations and business logic. MySQL is used as the database for storing event details, user records, registrations, and notifications.

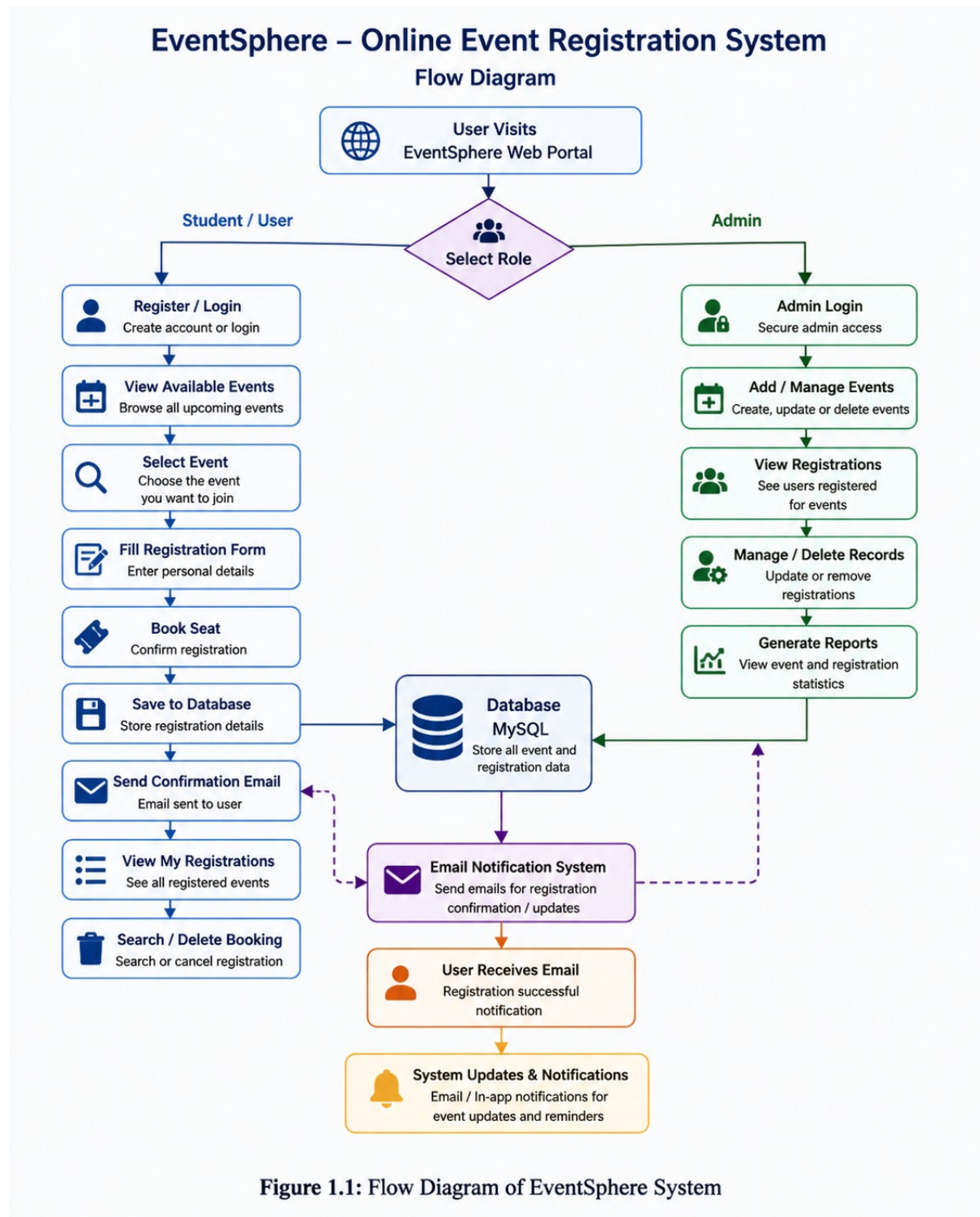


Figure 1.1: System Overview

The above figure illustrates the overall architecture of the Smart Campus Event Management System.

It shows how different users such as students, organizers, and administrators interact with the system. The user interface allows users to register, view events, and manage event-related activities.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several event management platforms are available in the market, such as Eventbrite, Cvent, and Meetup. These applications provide event registration, scheduling, and participant management features. However, they are designed for general-purpose event management and may not fully meet the specific needs of academic institutions.

Eventbrite is widely used for public event ticketing and registration, but it lacks campus-specific modules such as faculty approvals and student-based event categorization. Meetup is more community-focused and suitable for informal gatherings rather than structured academic events. Cvent provides enterprise-level event management but is expensive and complex for educational institutions.

The Smart Campus Event Management System is specifically designed for campus environments. It provides customized features such

as student registration, academic event categorization, faculty approval workflows, and centralized campus communication, making it more suitable for colleges and universities.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend environment is developed using HTML, CSS, and JavaScript. A code editor such as Visual Studio Code is used for development. Bootstrap can also be used for responsive design and improved UI components.

3.1.2 Structure of the Project

The frontend contains pages such as Home, Login, Registration, Event Listing, Event Details, Dashboard, and Contact Page. CSS files are used for styling, and JavaScript is used for form validation and dynamic interactions.

3.1.3 Execution Procedure

The frontend files are executed in a browser using a local server environment such as XAMPP. Users can access the application through

localhost and interact with the UI for registration and event browsing.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using PHP and executed using XAMPP server. PHP handles form processing, authentication, event management, and database connectivity.

3.2.2 Structure of the Project

The backend consists of modules for authentication, event creation, event approval, user registration, notification handling, and admin management.

3.2.3 Execution Procedure

The backend logic is connected to the frontend through PHP scripts. User requests are processed on the server, validated, and stored in the database.

3.3 Database Implementation

3.3.1 Database Configuration

MySQL is used as the database system. The database is configured using phpMyAdmin in XAMPP.

3.3.2 Schema Structure

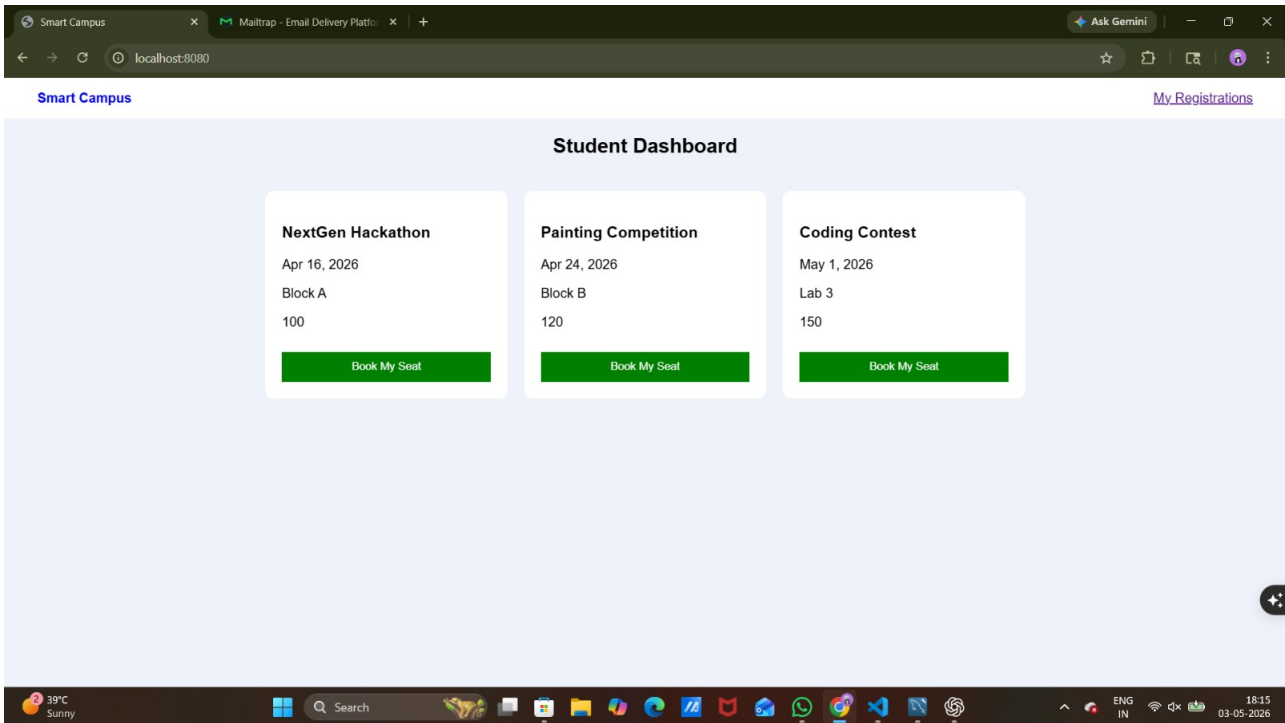


Figure 3.1: Frontend and Backend Workflow

Figure 3.1: Frontend and Backend Workflow The above figure illustrates the frontend and backend workflow of the Smart Campus Event Management System. It shows how users interact with the system through the user interface to access various features. Students can view available events, register for events, and track their participation status.

The database schema includes tables such as users, events, registrations, notifications, and admins.

3.3.3 Tables created

The tables created in the database are:

- Users Table
- Events Table
- Registrations Table
- Notifications Table

- Admin Table

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The system follows a three-tier architecture consisting of presentation layer, application layer, and database layer. The presentation layer handles the user interface, the application layer processes logic, and the database layer stores and retrieves data.

4.2 DataFlow Diagram

The data flow begins with user login and authentication. Event organizers create events, students register, administrators approve requests, and notifications are sent to users. Data is continuously exchanged between the frontend, backend, and database.

4.3 Module 1: User Management

This module handles student and organizer registration, login, profile management, and authentication. The User Management module in the Smart Campus Event Management System handles all user-related activities. It allows students, event organizers, and administrators to register in the system. Users can log in securely using their credentials to access the platform. The module ensures proper authentication and authorization of users. Students can view and register for events through their accounts. Event organizers can create and manage events after logging in. Administrators can monitor users and control system activities. Users can update their profiles and manage their personal details. Role-based access ensures that each user has specific permissions. This module improves security, organization, and efficient user interaction within the system.

4.4 Module 2: Event Management

This module allows event creation, event editing, event deletion, event scheduling, and event approval. The Event Management module in the Smart Campus Event Management System handles all event-related activities. It allows organizers to create and publish events within the campus. Organizers can edit event details such as date, The Event

Management module in the Smart Campus Event Management System handles all event- related activities. It allows organizers to create and publish events within the campus. Organizers can edit event details such as date,time, venue, and description. The module provides options to delete or update events when required. Students can view available events and register for them . It supports event scheduling to avoid clashes between multiple events. Administrators can approve or reject events created by organizers. The system maintains all event information in an organized manner. This module helps in efficient planning and co- ordination of campus events. Overall, it improves event management and enhances student participation.

4.5 Module 3: Notification Management

This module manages notifications, reminders, announcements, and updates related to events.The Notification Management module in the Smart Campus Event Management System handles all user notifications. It sends updates to users about upcoming events and important announcements. Students receive notifications about event registrations and schedules. Organizers are notified about event approvals and participant updates. The module supports reminders for events to avoid missing important activities. Notifications can be sent through the

system interface or alerts. It ensures timely communication between users and administrators. The system keeps users informed about any changes in event details. This module improves user engagement and participation in events. Overall, it provides effective communication within the system.

Advantages of this Project

The project offers a centralized platform for event coordination, improves communication, reduces manual work, and increases event participation.

a) Efficient Scheduling: Prevents event clashes and improves planning.

b) Centralized Access: All event details are available in one platform.

c) Easy Registration: Students can register quickly and receive updates.

Disadvantages

The system depends on internet connectivity and requires basic technical knowledge for operation.

Chapter 5

RESULTS AND DISCUSSIONS

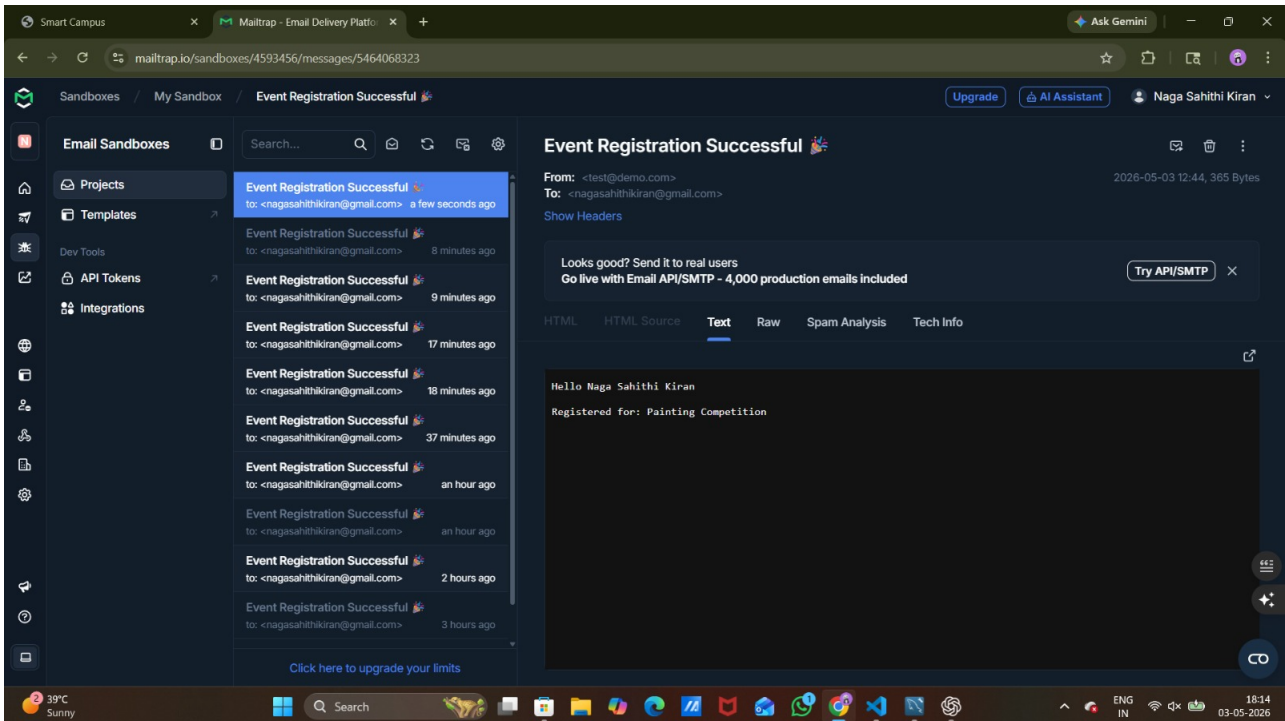
The EventSphere - Online Event Registration System was successfully developed and tested. The system allows smooth event creation, online registration, approval workflows, and notification delivery. Testing results show improved efficiency, reduced manual errors, and better event coordination compared to traditional manual methods.

Table 5.1: Comparison of Existing System and Proposed System

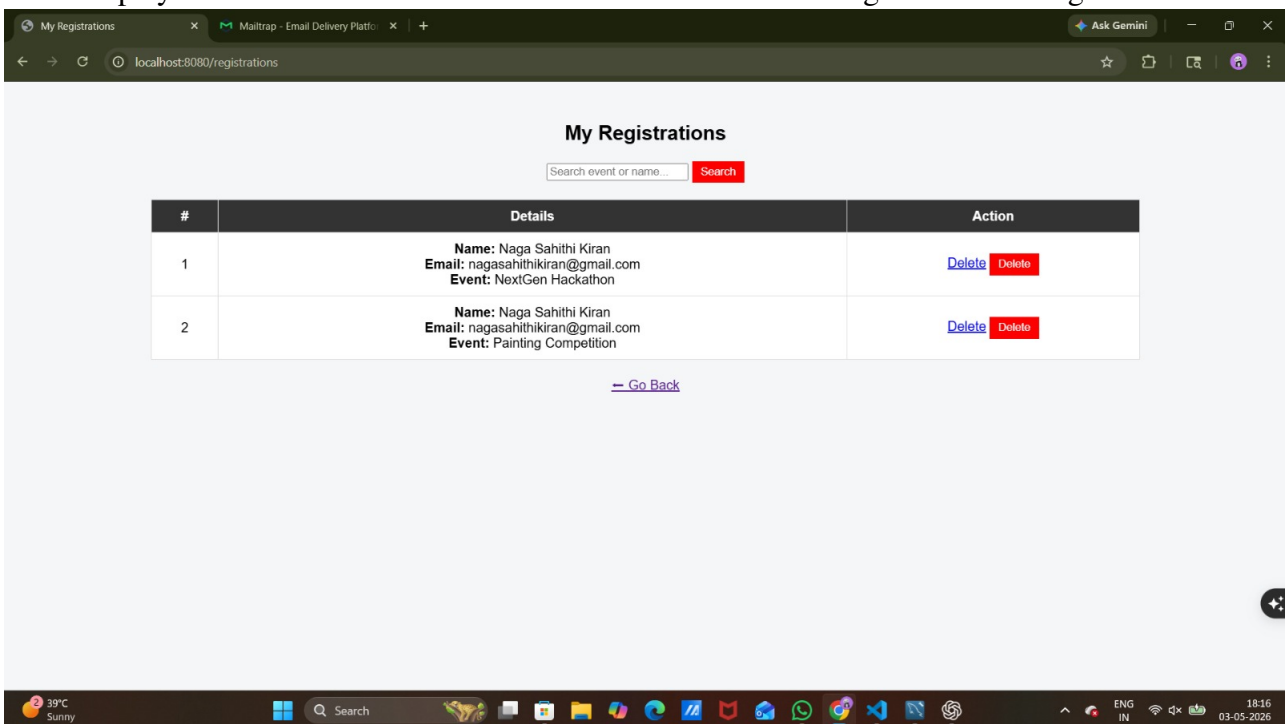
Features	Existing System	Proposed System
Event Registration	Manual	Online
Notifications	Delayed	Instant
Scheduling	Manual	Automated
Approval Process	Offline	Digital
Data Management	Paper Based	Centralized Database

The above table presents a comparison between the existing system and the proposed Smart Campus Event Management System. In the existing system, event registration is performed manually, which con-

sumes more time and effort. Notifications are delayed, and scheduling is handled manually, leading to inefficiencies. The approval process is offline, and data management is paper-based, making it less reliable.



The above figure shows the output screens of the Smart Campus Event Management System. It displays the user interface where students can view and manage their event registrations.



The above figure shows the registration details screen of the Smart Campus Event Management System. It displays the list of events for which the user has registered.

Figure 5.1: System Output Screens

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The EventSphere - Online Event Registration System provides an efficient and reliable solution for managing campus events digitally. It simplifies event organization, improves communication, reduces paperwork, and enhances the overall efficiency of event management in educational institutions.

6.2 Future Enhancements

Future enhancements include mobile app integration, AI-based event recommendations, QR-based attendance tracking, online payment support, chatbot assistance, and cloud deployment for scalability.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of knowledge	Requires in depth engineering knowledge	WK3,WK4, WK5, WK6	Requires knowledge in Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies (Thymeleaf, HTML/CSS) for developing a scalable event management system.
WP2	Conflicting requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and security by ensuring easy student access while maintaining secure admin authentication and data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Involves system design, database schema creation, validation handling, and efficient event filtering and registration mechanisms.
WP4	Familiarity	Partially new problems	WK4, WK8	Integrating real-time event updates, user interaction, and admin analytics introduces challenges beyond traditional manual systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for event filtering, registration tracking, and statistics generation not fully covered by standard templates.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Addresses requirements of students, faculty coordinators, and administrators ensuring smooth event organization and participation.
WP7	Interdependence	System level integration	WK3, WK5, WK6	Integrates frontend UI, backend services, database, validation, and security modules into a unified system.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system, extending beyond traditional event handling methods.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized event management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through authentication and controlled access.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management

Table 7.3: SDG Mapping for the Project

Table 7.1 – CEP (Complex Engineering Problem) The above table describes how the Smart Campus Event Management System addresses complex engineering problems. It highlights the need for in-depth technical knowledge such as Spring Boot, MVC architecture, and database design. The system balances both technical and non-technical requirements like usability and security.

Table 7.2 – CEA (Complex Engineering Activities) The above table explains how the project performs complex engineering activities. It uses a wide range of technologies such as Spring Boot, Spring MVC, Thymeleaf, and relational databases

Table 7.3 – SDG Mapping The above table shows the alignment of the project with Sustainable Development Goals (SDGs). It supports Quality Education by encouraging student participation in academic events.

References

- [1] Eventbrite, “Eventbrite Platform.” Available:
<https://www.eventbrite.com>
- [2] Meetup, “Meetup Events Platform.” Available:
<https://www.meetup.com>
- [3] Cvent, “Cvent Event Management Software.” Available:
<https://www.cvent.com>
- [4] PHP Documentation. Available: <https://www.php.net>
- [5] MySQL Documentation. Available: <https://www.mysql.com>

SOURCECODE

7.1 Source code

```
import org.springframework.stereotype.Controller ;
import org.springframework.ui.Model ;
import org.springframework.web.bind.annotation.* ;

import com.campus.eventsystem.model.Booking ;
import com.campus.eventsystem.repository.BookingRepository ;

import java.util.* ;

import org.springframework.mail.SimpleMailMessage ;
import org.springframework.mail.javamail.JavaMailSender ;
import org.springframework.beans.factory.annotation.Autowired ;

@Controller public class

    EventController {

        @Autowired private BookingRepository
            bookingRepository ;

        @Autowired private JavaMailSender
            mailSender ;

        //===== HOME =====
        @GetMapping ( "/" ) public String home(

            Model model ) {

            List <Map<String , String >> events = new ArrayList <>();

            Map<String , String > e1 = new HashMap<>(); e1 . put (
                "name" , "NextGen Hackathon " ); e1 . put ( " date " , "Apr
                16 , 2026" ); e1 . put ( " place " , " Block A" ); e1 . put ( "
                price " , "100" );

            Map<String , String > e2 = new HashMap<>(); e2 . put (
                "name" , "Painting Competition " ); e2 . put ( " date " , "Apr
                "
            )
        }
    }
}
```

```

24 , 2026" ); e2 . put ( " place " , " Block B" ); e2 . put ( "
price " , "120" );

Map<String , String > e3 = new HashMap<>(); e3 . put (
"name" , "Coding Contest " ); e3 . put ( " date " , "May 1 ,
2026" ); e3 . put ( " place " , "Lab 3" ); e3 . put ( " price " ,
"150" );

events . add ( e1 ); events . add ( e2 ); events . add ( e3 );
model . addAttribute ( " events " , events ); return " index " ;
}

//===== FORM =====
@GetMapping ( " / book-form " ) public String showForm ( @RequestParam
String event , Model model ) { model . addAttribute ( " event " , event ); return
"book-form" ;
}

//===== SAVE + EMAIL =====
@GetMapping ( " / book-seat " ) public String bookSeat (
@RequestParam String name ,
@RequestParam String email ,
@RequestParam String event ) {

// SAVE TO DATABASE
Booking booking = new Booking ( ) ; booking . setName
( name ) ; booking . setEmail ( email )

; booking . setEvent ( event ) ; bookingRepository

. save ( booking ) ;

// EMAIL try {
SimpleMailMessage message = new SimpleMailMessage ( ) ; message . setFrom ( "test@demo . com" )
; message . setTo ( email ) ; message . setSubject ( " Event Registration Successful
" ); message . setText ( " Hello " + name + "\n\nRegistered" + event );

mailSender . send ( message );
} catch ( Exception e ) {
System . out . println ( " Email failed : " + e . getMessage ( ) );
}

```

```

        return "success";
    } //===== SHOW

    =====

    @GetMapping ( "/registrations" ) public String showRegistrations ( Model
    model ) { model . addAttribute ( "bookings" , bookingRepository . findAll ( ) ) ; return "r
    egistrations" ;
    }

    //===== DELETE =====
    @GetMapping ( "/delete" ) public String deleteBooking (

    @RequestParam Long id ) { bookingRepository . deleteById ( id ) ; return
    "redirect:/registrations" ;
    }

    //===== CLEAR =====
    @GetMapping ( " / c l e a r " ) public String
    clearBookings ( ) { bookingRepository . deleteAll ( )

    ; return "redirect:/registrations" ; } }
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org

/POM/4.0.0■https://maven.apache.org/xsd<modelVersion>4.0.0</modelVersion>

<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
</parent>

<groupId>com.campus</groupId>
<artifactId>eventsystem</artifactId>
<version>0.0.1-SNAPSHOT</version>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>

```

```

<!-- WEB -->
<dependency>
    <groupId>org . springframework . boot </groupId>
    <artifactId>spring -boot -starter -web</artifactId>
</dependency>
<!-- THYMELEAF -->
<dependency>
    <groupId>org . springframework . boot </groupId>
    <artifactId>spring -boot -starter -thymeleaf </artifactId>
</dependency>

<!-- MAIL -->
<dependency>
    <groupId>org . springframework . boot </groupId>
    <artifactId>spring -boot -starter -mail </artifactId>
</dependency>

<!-- JPA -->
<dependency>
    <groupId>org . springframework . boot </groupId>
    <artifactId>spring -boot -starter -data -jpa </artifactId>
</dependency>

<!-- MYSQL -->
<dependency>
    <groupId>com . mysql </groupId>
    <artifactId>mysql -connector -j </artifactId>
    <scope>runtime </scope>
</dependency>

<!-- TEST -->
<dependency>
    <groupId>org . springframework . boot </groupId>
    <artifactId>spring -boot -starter -test </artifactId>
    <scope>test </scope>
</dependency>

</dependencies>

<build >
    <plugins >

```

```

        <plugin>
            <groupId>org . springframework . boot </groupId>
            <artifactId>spring -boot -maven -plugin </artifactId> </plugin>
        </plugins>
    </build>

</project>
spring . mail . host=sandbox . smtp . mailtrap . io
spring . mail . port =2525 spring .
    mail . username=6ab4511b3c4d64 spring .
    mail .
password=4e0be41003e861

spring . mail . properties . mail . smtp . auth= true spring . mail . prop e
rties . mail . smtp . starttls . enable= true spring . datasource . url =jdbc : mysql : //

localhost :3306/ eventsystem spring . datasource

. username= root spring . datasource . password= sahithi2814

spring . jpa . hibernate . ddl -auto=update spring . jpa
. show - sql= true package com . campus . eventsystem
. model ;

import jakarta . persistence . * ;
@Entity public class
Booking {

    @Id
    @GeneratedValue ( strategy = GenerationType . IDENTITY) private
    Long id ;

    private String name ; private
    String email ; private
    String event ;
    public Long getId () { return id ;
}

```

```

    public String getName () { return name ;
    }

    public void setName ( String t h name ) {
        i s . name = name ;
    } public String getEmail () { return email

;

}

                                email ) {

    public void setEmail ( String t h i

        s . email = email ;
    } public String getEvent () { return event

;

}

    public void setEvent ( String event ) {
        t h i s . event = event ;
    }
} package com . campus . eventsystem ;
import org . springframework . boot . SpringApplication ; import org . springframework . boot

. autoconfigure . SpringBootApplication ;

@SpringBootApplication public c l a s s

    EventsystemApplication {

        public static void main ( String [] args ) {
            SpringApplication . run ( EventsystemApplication . class , args ) ;
        }
    }
<!DOCTYPE html>
<html xmlns : th=" http : / /www. thymeleaf . org ">
<head>
    <t i t l e >Smart Campus</ t i t l e >

    <style > body { font -family : Arial ;
        margin : 0; background : #
        eef3fb ;
    }

```

```

    . navbar { display : flex ; j u s t i f y – content : space –between
                ; padding : 15px 40px ; background
                : white ;
    }

    . logo { font –weight : bold ; color :
            blue ;
    }

    . cards { display : flex ; j u s t i f y – content
            : center ; gap : 20px ; margin –top :
            40px ;
    }

    . card { background : white ; padding : 20px
            ; width : 250px ; border – radius :
            10px
            ;
    }

    . btn { margin –top : 10px ;
            background : green ; color
            : white ; padding : 10px ;
            border : none ; width :
            100% ;
    }
</ style >
</ head >

< body >

< div      c l a s s = " navbar " >
    < div      c l a s s = " logo " > Smart Campus </ div >
    < a href = " / r e g i s t r a t i o n s " > My Registrations </ a> </ div >

< h2      s t y l e = " text – align : center ; " > Student      Dashboard </ h2 >

< div      c l a s s = " cards " >

    < div      c l a s s = " card "      t h : each = " event " : { events } >

```



```

<h3      th : t e x t ="${ event . name }"></h3>
<p th : t e x t ="${ event . date }"></p>
<p th : t e x t ="${ event . place }"></p>
<p th : t e x t ="${ event . price }"></p>

<a th : href ="@{/ book-form ( event=${ event . name })}">
    <button      c l a s s =" btn ">Book My Seat </ button>
</a>
</div>

```

```

</div>

```

```

</body>

```

```

</html>

```

```

<!DOCTYPE html>

```

```

<html      xmlns : th =" http : / /www . thymeleaf . org ">

```

```

<head>

```

```

<t i t l e >My Registrations </ t i t l e >

```

```

<style > body { font -family : Arial ;
background : # f4f6f8 ; padding :
20px ; }

```

```

h2 {
text - align :      center ;
}

```

```

t a b l e { width : 80% ; margin : auto ; border
- collapse : collapse ; background : white ; }

```

```

th ,      td {
padding : 10px ; border : 1px s o l i
d #ddd ; text - align : center ;
} th { background : #333 ; color

```

```

:

```

```

white ; }

```

```

button { padding : 5px 10px ;
background : red ; color :
white ; border : none ;
cursor : pointer ;

```

```

    }

    . search -box {
        text - align : center ; margin

        -bottom : 20px ;
    }
</ style >
</head>

<body>

<h2>My Registrations </h2>

<div class=" search -box">
    <form action ="/ search ">
        <input type=" t e x t " name="keyword" placeholder =" Search ■ event ■ or ■ name . . . " required >

        <button type=" submit ">Search </ button>
    </form>
</div>

<table >
    <tr >
        <th >#</th>
        <th>Details </th>
        <th>Action </th>
    </ tr >

    <tr th : each=" booking , ■ s t a t . ■ ${ bookings }">
        <td th : t e x t ="${ s t a t . index + 1 }"></td>
        <td>

            <b>Name: </b><span th : t e x t ="${ booking . name }"></span><br>
            <b>Email : </b><span th : t e x t ="${ booking . email }"></span><br>
            <b>Event : </b><span th : t e x t ="${ booking . event }"></span>

        </td>
        <td>
            <a th : href ="@/{ delete ( id=${ booking . id }} ) ">Delete </a>
            <button>Delete </ button>
        </a>
        </td>
    </tr>

```

```
        </ tr >
</ table >
<br>
<div      s t y l e =" text - align : center ; ">
    <a    href =" / ">      Go Back</a>
</div>

</body>
</html>
```